

sandy.

The Sandy Programming Language

As easy as Python. Compiles to a native binary. Checked before you ship.

Santhosh Arulsamy

A complete guide to the language,
its standard library, and its tools.

Contents

1. Chapter 1 — Introduction
2. Chapter 2 — Getting Started
3. Chapter 3 — Lexical Structure
4. Chapter 4 — Values and Types
5. Chapter 5 — Variables and Assignment
6. Chapter 6 — Operators and Expressions
7. Chapter 7 — Strings
8. Chapter 8 — Control Flow
9. Chapter 9 — Functions
10. Chapter 10 — Collections
11. Chapter 11 — Structs
12. Chapter 12 — Error Handling
13. Chapter 13 — Modules and Packages
14. Chapter 14 — The Type System
15. Chapter 15 — Concurrency
16. Chapter 16 — Compilation and Performance
17. Chapter 17 — The Standard Library
18. Chapter 18 — Built-in Functions
19. Chapter 19 — Tooling and the CLI
20. Chapter 20 — Appendix: Grammar

Chapter 1 — Introduction

What Sandy is

Sandy is a general-purpose programming language. Programs are stored in files ending in `.sy`. It looks and feels like a modern scripting language — clean syntax, dynamic by default, batteries included — but it is built around one idea that scripting languages usually give up on: **the typed parts of your program can be compiled to a small, fast, standalone native executable.**

That single idea shapes everything:

- You can start writing with no ceremony — no type declarations, no build step, no boilerplate `main`.
- You can add type annotations gradually, and a checker verifies them *before* the program runs, turning a class of bugs into compile-time errors.
- When a piece of code needs to be fast, you compile it. Typed Sandy lowers to C and runs at C speed — many times faster than an interpreter.

Why it exists

Every popular language is a set of tradeoffs. Python is wonderfully easy but slow and dynamically checked. C and Rust are fast and safe but demand ceremony. Most projects pick one language and live with its weak side.

Sandy aims at a specific, defensible lane between them:

As easy to write as Python, but it compiles to a fast native binary, tells you exactly what's wrong before you ship, and needs zero setup to run.

It does not try to be the best at everything — no language is. It tries to be the best **easy language that produces a fast native binary with types checked up front.** Python cannot produce that binary; C and Rust are not this easy.

The four goals

Sandy is designed around four goals, pursued in a deliberate order because at their extremes they pull against each other:

Goal	What it means
Easy	Python-level readability, minimal ceremony

Goal	What it means
Fast	Native-code speed for the typed core
Safe	Catch mistakes before the program runs
Lovable	Great errors, good tools, zero configuration

The key design decision — **gradual types** — serves three of these at once: types keep the language *easy* (they're optional), make it *safe* (they're checked early), and make it *fast* (the compiler uses them to generate unboxed native code).

A first taste

Here is a complete Sandy program:

```
fn greet(name: string) -> string {
    return "hello, " + name
}

for who in ["world", "Sandy"] {
    print(greet(who))
}
```

```
hello, world
hello, Sandy
```

Notice what is *not* there: no imports for basic output, no `main` function, no semicolons, no manual memory management. The `: string` and `-> string` annotations are optional — the program runs identically without them — but with them, the checker verifies every call to `greet` before the program starts.

How Sandy runs your code

Sandy has three ways to execute a program, all from the same source:

1. **The interpreter** (`sandy run`) — a tree-walking evaluator. Instant, no build step. This is where you develop.
2. **The bytecode VM** (`sandy --vm run`) — compiles to bytecode and runs it on a stack machine; a faster interpreter, kept behavior-identical to the first.
3. **The native compiler** (`sandy build`) — translates the typed subset to C and compiles it to a real executable that runs at C speed.

The interpreter and VM run *all* of Sandy, including its most dynamic features. The native compiler runs the statically typed subset — which is most of the language — and is where the speed comes from.

An honest word

Because the interpreter and VM are themselves written in Python, they run Sandy programs *slower* than CPython runs Python. They exist for instant feedback while you develop, not for raw throughput. The speed advantage — 15 to 180 times faster than CPython on compute-bound code — comes from the **native compiler**. Develop against the interpreter; ship the compiled binary.

The rest of this book shows you the whole language. The next chapter gets you running code in a few minutes.

Chapter 2 — Getting Started

Installing

Sandy needs Python 3.11 or newer. Clone the repository and run it as a module — there is nothing to compile or install:

```
git clone https://github.com/santhosharulsamy/Sandy.git
cd Sandy
python -m sandy --version
```

To build native binaries you also need a C compiler on your `PATH` — `cc`, `gcc`, or `clang`. Almost every machine already has one.

Your first program

Create a file `hello.sy`:

```
print("Hello, Sandy!")
```

Run it:

```
python -m sandy run hello.sy
```

```
Hello, Sandy!
```

There is no `main` function and no boilerplate. Top-level statements run top to bottom.

The four commands you'll use most

<code>python -m sandy run hello.sy</code>	<code># run on the interpreter</code>
<code>python -m sandy check hello.sy</code>	<code># type-check without running</code>
<code>python -m sandy build hello.sy</code>	<code># compile to a native binary ./hello</code>
<code>python -m sandy</code>	<code># start an interactive REPL</code>

In the REPL you can type expressions and see their values immediately — handy for experimenting as you read this book.

A whirlwind tour

The rest of this chapter is the entire language in miniature. Don't worry about absorbing it all; each feature gets its own chapter later. Skim it to get the shape of the language.

Variables and printing

```
name = "Ada"
age = 36
print("{name} is {age}")      # {name} is 36
```

Assign to declare. Strings interpolate with `{...}`.

Numbers

```
print(7 + 2)      # 9
print(7 / 2)      # 3.5  - division is always a float
print(7 % 2)      # 1
print(2 ** 8)     # 256
```

Functions

```
fn add(a, b) {
    return a + b
}
print(add(2, 3))  # 5
```

Optional types

```
fn area(w: int, h: int) -> int {
    return w * h
}
```

Add annotations and `sandy check` verifies them before running. Leave them off and the code is fully dynamic.

Control flow

```
for i in range(3) {
    if i % 2 == 0 { print("{i} even") } else { print("{i} odd") }
}
```

```
0 even
1 odd
```

```
2 even
```

Lists and maps

```
xs = [1, 2, 3]
push(xs, 4)
print(xs)                # [1, 2, 3, 4]

scores = {"ada": 10, "bob": 7}
print(scores["ada"])     # 10
```

Structs

```
struct Point { x: int, y: int }
p = Point(1, 2)
p.y += 5
print(p)                 # Point(x=1, y=7)
```

Errors

```
try {
  x = int("not a number")
} catch e {
  print("failed: " + e)    # failed: cannot convert 'not a number' to int
}
```

Modules and the standard library

```
import "math" as math
print(math.gcd(12, 18))   # 6
```

Concurrency

```
ch = channel()
fn worker() { send(ch, 42) }
spawn(worker)
print(recv(ch))           # 42
```

Compiling to native


```
fn fib(n: int) -> int {  
    if n < 2 { return n }  
    return fib(n - 1) + fib(n - 2)  
}  
print(fib(30))
```

```
python -m sandy build fib.sy --run
```

```
built native executable: fib  
832040
```

That is the whole language on one page. The chapters that follow take each piece apart carefully, starting with the smallest — how Sandy reads your source text.

Chapter 3 — Lexical Structure

Before Sandy understands your program, it breaks the source text into *tokens*: names, keywords, numbers, strings, operators, and punctuation. This chapter describes those raw ingredients and the small number of rules about whitespace and comments.

Source files

A Sandy program is a UTF-8 text file, conventionally ending in `.sy`. A program is a sequence of statements. There is no required top-level function; execution begins at the first statement.

Statements end at newlines

Sandy separates statements by **newlines**, not semicolons. There is no `;` statement terminator, and you cannot put two statements on one line with a separator:

```
x = 1
y = 2          # correct: one statement per line
```

Blank lines and indentation are insignificant except as readability — Sandy is brace-delimited, not indentation-delimited. Blocks are written with `{ }`.

Comments

A comment starts with `#` and runs to the end of the line:

```
# this is a comment
x = 42          # so is this
```

There are no block comments; use `#` on each line.

Identifiers

An identifier names a variable, function, struct, parameter, or module. It starts with a letter or underscore and continues with letters, digits, or underscores:

```
count  _total  isReady  fib2  snake_case_name
```

By convention, a leading underscore (`_helper`) marks something internal.

Keywords

The following words are reserved and cannot be used as identifiers:

```
fn    return    if    elif    else    while    for    in    break    continue
try   catch    throw  struct  import  true    false  nil    and    or    not
```

`as` is a *soft* keyword: it has special meaning only in `import "x" as name`, and is otherwise an ordinary identifier.

Literals

Integers are written in the usual way. They have arbitrary precision on the interpreter and VM (they are Python integers), and 64-bit range when compiled natively.

```
0      42      1000000
```

Floats contain a decimal point:

```
3.14    0.5    10.0
```

Booleans are `true` and `false`. **Nil**, the absence of a value, is `nil`.

Strings are enclosed in double quotes and may contain escape sequences and interpolation (Chapter 7):

```
"hello"
"line one\nline two"
"total: {count}"
```

List and map literals use brackets and braces:

```
[1, 2, 3]
{"a": 1, "b": 2}
```

Operators and punctuation

The operator and punctuation tokens are:

<code>+</code>	<code>-</code>	<code>*</code>	<code>/</code>	<code>%</code>	<code>**</code>	arithmetic
<code>==</code>	<code>!=</code>	<code><</code>	<code>></code>	<code><=</code>	<code>>=</code>	comparison
<code>=</code>	<code>+=</code>	<code>-=</code>	<code>*=</code>	<code>/=</code>		assignment

() [] { }	grouping, indexing, blocks/maps
, : . ->	separators and annotations

The words `and`, `or`, and `not` are the logical operators (Chapter 6).

A note on braces inside strings

Because `{` begins interpolation inside a string, a *literal* brace in a string is written by doubling it: `"{{"` produces `{` and `"}}"` produces `}`. This only matters inside string literals; everywhere else braces delimit blocks and maps as usual. See Chapter 7.

With the tokens established, the next chapter describes the values those tokens denote.

Chapter 4 — Values and Types

Every value in Sandy has a type. This chapter surveys the built-in types; later chapters cover each in depth. The `type(x)` builtin returns a value's type name as a string, which is a good way to explore in the REPL.

The type names

Type	Example values	<code>type(x)</code>
nil	<code>nil</code>	<code>"nil"</code>
bool	<code>true</code> , <code>false</code>	<code>"bool"</code>
int	<code>0</code> , <code>42</code> , <code>-7</code>	<code>"int"</code>
float	<code>3.14</code> , <code>10.0</code>	<code>"float"</code>
string	<code>"hi"</code>	<code>"string"</code>
list	<code>[1, 2, 3]</code>	<code>"list"</code>
map	<code>{"a": 1}</code>	<code>"map"</code>
struct instance	<code>Point(1, 2)</code>	the struct's name, e.g. <code>"Point"</code>
function	<code>fn f() {}</code>	<code>"function"</code>

Concurrency values add two more: a `channel` and a `task` (Chapter 15).

Nil

`nil` is the absence of a value. Functions that don't return anything yield `nil`, and `recv` on a closed, drained channel returns `nil`. `nil` is *falsy* (Chapter 6).

Booleans

`true` and `false`. They are produced by comparisons and consumed by conditions and the logical operators.

Numbers: int and float

Sandy has two numeric types. Integers are whole numbers; floats have a fractional part. The important rule to internalize early:

Division / always produces a float. `10 / 2` is `5.0`, not `5`.

```
print(type(10 + 2))    # int
print(type(10 / 2))    # float
print(type(10.0))      # float
```

Mixed arithmetic (`int` with `float`) produces a `float` . Convert explicitly with `int(x)` and `float(x)` :

```
print(int(3.9))        # 3    (truncates toward zero)
print(float(3))         # 3.0
```

Integers on the interpreter and VM have arbitrary precision. When you compile a program natively, integers are 64-bit — a deliberate tradeoff for speed.

Strings

A string is an immutable sequence of characters. String operations produce new strings; they never modify the original. Strings support concatenation, indexing, iteration, interpolation, and a set of methods — all of Chapter 7.

```
s = "hello"
print(len(s))          # 5
print(s[0])            # h
print(s.upper())        # HELLO
```

Lists

A list is an ordered, growable, mutable sequence. Elements may be of any type, though the native compiler requires them to be homogeneous (all the same type).

```
xs = [10, 20, 30]
push(xs, 40)
print(xs[1])           # 20
print(xs[-1])          # 40    (negative indexes count from the end)
```

Lists are covered in Chapter 10.

Maps

A map associates keys with values and preserves insertion order. Keys are typically strings or integers.

```
m = {"one": 1, "two": 2}
m["three"] = 3
print(m["two"])      # 2
print(has(m, "four")) # false
```

Maps are covered in Chapter 10.

Structs

A struct is a user-defined type: a named bundle of fields. Struct instances are compared by value and passed by reference.

```
struct Point { x: int, y: int }
p = Point(3, 4)
print(p.x)      # 3
```

Structs are covered in Chapter 11.

Functions are values

A function is a first-class value. You can store it in a variable, pass it as an argument, and return it from another function.

```
fn square(n) { return n * n }
f = square
print(f(9))      # 81
```

Functions are covered in Chapter 9.

Dynamic by default, typed by choice

Sandy is dynamically typed: a variable can hold any type, and can be reassigned to a value of a different type. The optional type annotations you saw in Chapter 2 do not change how values behave at runtime — they add checks that run *before* the program does. The type system is the subject of Chapter 14.

```
x = 5
x = "now a string"  # fine — x is dynamic
print(x)           # now a string
```

The next chapter covers how names get bound to these values.

Chapter 5 — Variables and Assignment

Declaring by assigning

There is no separate declaration keyword. Assigning to a name for the first time creates it:

```
count = 0
message = "ready"
```

A name can be reassigned freely, including to a value of a different type (Sandy is dynamically typed):

```
x = 10
x = "ten"      # allowed
```

Optional type annotations

You may annotate a variable with a type at the point of first assignment:

```
count: int = 0
name: string = "Sandy"
scores: map<string, int> = {}
```

The annotation is checked by `sandy check` and by the compiler: a later assignment of an incompatible type is a type error (Chapter 14). Without an annotation, the variable is dynamic.

Compound assignment

The compound assignment operators update a variable in place:

```
total = 0
total += 5      # total = total + 5
total -= 2      # subtract
total *= 3      # multiply
total /= 2      # divide (result is a float)
```

Compound assignment requires the variable to already exist.

Because `/=` uses division, it produces a float:

```
x = 10
x /= 2
```



```
print(x)          # 5.0
```

Scope

Sandy uses **function-level scope**. Names assigned at the top level of a file are global to that file. Names assigned inside a function are local to that function. A block (`{ }`) attached to `if`, `while`, or `for` does *not* introduce a new scope — a variable first assigned inside a loop is visible after it:

```
for i in range(3) {  
    last = i  
}  
print(last)      # 2 - `last` is visible here
```

Inside a function, assigning to a name creates or updates a *local* variable; it does not modify a global of the same name. Functions can *read* enclosing variables (see closures, Chapter 9), but a plain assignment inside a function binds locally.

Indexed and field assignment

Assignment also targets a list element, a map entry, or a struct field:

```
xs = [1, 2, 3]  
xs[0] = 99      # list element  
  
m = {"a": 1}  
m["b"] = 2      # map entry (creates it)  
  
struct P { x: int }  
p = P(1)  
p.x = 42        # struct field
```

These do not create a new variable; they mutate the existing container or instance.

The value of assignment

Assignment is a statement, not an expression — it does not produce a value you can use inline. This rules out the classic `if (x = f())` mistake; write the assignment on its own line.

The next chapter covers the operators that combine these values into expressions.

Chapter 6 — Operators and Expressions

An expression computes a value. This chapter covers every operator, how they combine, and the precedence that decides who binds first.

Arithmetic

Operator	Meaning	Note
<code>+</code>	addition	also concatenates strings and lists
<code>-</code>	subtraction	also unary negation
<code>*</code>	multiplication	also repeats strings/lists
<code>/</code>	division	always produces a float
<code>%</code>	modulo	Python-style <i>floor</i> modulo
<code>**</code>	exponentiation	right-associative

```
print(2 + 3)      # 5
print(2 ** 3 ** 2) # 512  - ** is right-associative: 2 ** (3 ** 2)
print(7 / 2)      # 3.5
print(-7 % 3)     # 2      - result has the sign of the divisor
print(7 % -3)     # -2
```

Arithmetic between an `int` and a `float` yields a `float`. Division always yields a `float`, even for evenly divisible integers (`9 / 3` is `3.0`).

There is no integer-division operator; use `floor(a / b)` when you need one.

Comparison

`==`, `!=`, `<`, `>`, `<=`, `>=` compare two values and produce a bool. Numbers compare numerically and strings lexicographically. Structs, lists, and maps compare by value with `==`:

```
print(3 < 5)      # true
print("apple" < "banana") # true
print([1, 2] == [1, 2]) # true
```

Ordering (`<`, `>`, ...) is defined for numbers and strings. `==` and `!=` work on any two values.

Logical operators

`and`, `or`, and `not` are the boolean operators, and `and` / `or` **short-circuit** — the right operand is not evaluated if the left already decides the result:

```
print(true and false)    # false
print(false or "yes")    # yes
print(not nil)           # true
```

`and` returns its first falsy operand or the last operand; `or` returns its first truthy operand. Combined with truthiness (below), this gives the familiar `x or default` idiom.

Truthiness

A value used as a condition is tested for *truthiness*. These are **falsy**:

- `nil`
- `false`
- `0` and `0.0`
- the empty string `""`
- the empty list `[]` and empty map `{}`

Everything else is truthy.

String and list operators

`+` concatenates, and `*` repeats:

```
print("ab" + "cd")      # abcd
print("ab" * 3)          # ababab
print([1] + [2, 3])     # [1, 2, 3]
```

Indexing and calls

Postfix operators access parts of a value:

```
xs[0]          # list/string index (negative counts from the end)
m["key"]       # map lookup
p.field        # struct field or module member
f(a, b)        # function call
s.upper()      # method call
```

Precedence

From highest (binds tightest) to lowest:

1. Postfix: calls `f(...)`, indexing `[...]`, member `.`
2. Unary: `-x`, `not x`
3. `**` (right-associative)
4. `*`, `/`, `%`
5. `+`, `-`
6. Comparisons: `<`, `>`, `<=`, `>=`, `==`, `!=`
7. `and`
8. `or`

Use parentheses to group when in doubt; they always override precedence:

```
print(2 + 3 * 4)      # 14
print((2 + 3) * 4)    # 20
```

The next chapter looks closely at one of the most-used types: strings.

Chapter 7 — Strings

Strings are one of the most-used types, so Sandy gives them first-class syntax and a useful set of operations. A string is an **immutable** sequence of characters: every operation returns a new string.

Literals and escapes

String literals are enclosed in double quotes:

```
"hello"
```

Inside a literal, a backslash introduces an escape:

Escape	Meaning
<code>\n</code>	newline
<code>\t</code>	tab
<code>\r</code>	carriage return
<code>\"</code>	double quote
<code>\\</code>	backslash

```
print("line one\nline two")
print("she said \"hi\"")
```

Interpolation

A pair of braces inside a string embeds an expression, converted to a string:

```
name = "Ada"
count = 3
print("hello {name}, you have {count} items")
print("2 + 2 = {2 + 2}")
```

```
hello Ada, you have 3 items
2 + 2 = 4
```

Any expression works inside `{...}`. To include a **literal** brace, double it:

```
print("a set is written {{ }}")    # a set is written { }
```

This doubling rule is the one thing to remember when a string needs real braces (for example, when producing JSON by hand — though the `json` module is the better tool).

Concatenation and repetition

```
print("foo" + "bar")              # foobar
print("=" * 10)                   # =====
```

Only strings may be concatenated with `+`. To join a number, convert it first with `str`:

```
n = 42
print("answer: " + str(n))        # answer: 42
```

Length, indexing, and iteration

```
s = "hello"
print(len(s))                    # 5
print(s[0])                      # h
print(s[-1])                     # o
for ch in s:
    print(ch)                    # h e l l o, one per line
}
```

Indexing returns a one-character string. Negative indexes count from the end.

Comparison

Strings compare lexicographically by character code:

```
print("apple" < "banana")        # true
print("Zoo" < "apple")           # true  – uppercase sorts before lowercase
```

Methods

Strings carry a set of methods, called with `.`:

Method	Result
<code>s.upper()</code>	uppercase copy
<code>s.lower()</code>	lowercase copy

Method	Result
<code>s.trim()</code>	copy with leading/trailing whitespace removed
<code>s.length()</code>	length (same as <code>len(s)</code>)
<code>s.replace(a, b)</code>	copy with each <code>a</code> replaced by <code>b</code>
<code>s.starts_with(p)</code>	whether it begins with <code>p</code>
<code>s.ends_with(p)</code>	whether it ends with <code>p</code>
<code>s.has(sub)</code>	whether it contains <code>sub</code>
<code>s.split(sep)</code>	list of pieces (whitespace if <code>sep</code> omitted)

```
print(" Hello ".trim().upper()) # HELLO
print("a,b,c".split(","))      # ["a", "b", "c"]
print("readme.sy".ends_with(".sy")) # true
```

Many of these also exist as global builtins (`upper(s)`, `split(s, sep)`, `join(list, sep)` — Chapter 18), and character-level helpers plus `ord` / `chr` live in the `text` module (Chapter 17).

Converting to and from strings

`str(x)` converts any value to its display string; `int(s)` and `float(s)` parse numbers:

```
print(str(3.5))      # 3.5
print(int("42") + 1) # 43
```

The next chapter turns from data to control: how programs make decisions and repeat work.

Chapter 8 — Control Flow

Control-flow statements decide which code runs and how often. Sandy has three: `if`, `while`, and `for`. All use brace-delimited blocks.

`if / elif / else`

```
n = 7
if n < 0 {
    print("negative")
} elif n == 0 {
    print("zero")
} else {
    print("positive")
}
```

The condition is any expression, tested for truthiness (Chapter 6) — it need not be a bool. There may be any number of `elif` branches and an optional final `else`.

A syntax rule worth stating plainly: `elif` and `else` must appear on the *same line* as the preceding closing brace:

```
if x { ... } else { ... }      # correct

if x { ... }
else { ... }                  # WRONG — `else` must follow the `}`
```

This is because a newline ends the `if` statement. Keep `} elif` and `} else` together.

`while`

`while` repeats its block as long as the condition is truthy:

```
i = 0
while i < 5 {
    print(i)
    i += 1
}
```

```
0
1
2
```



```
3
4
```

for

for iterates over a sequence: a range, a list, or a map.

Over a range

range produces integers. It comes in three forms:

```
for i in range(3) { print(i) }      # 0, 1, 2
for i in range(1, 4) { print(i) }  # 1, 2, 3
for i in range(0, 10, 2) { print(i) } # 0, 2, 4, 6, 8
```

range(n) counts from 0 up to (not including) **n**. **range(a, b)** counts from **a** to **b**. **range(a, b, step)** uses a stride, which may be negative:

```
for i in range(3, 0, -1) { print(i) } # 3, 2, 1
```

Over a list

```
for name in ["Ada", "Bob", "Cy"] {
    print(name)
}
```

Over a map

Iterating a map yields its **keys**, in insertion order:

```
scores = {"ada": 10, "bob": 7}
for who in scores {
    print(who + ": " + str(scores[who]))
}
```

```
ada: 10
bob: 7
```

break and continue

break exits the nearest enclosing loop; **continue** skips to its next iteration:

```
total = 0
for i in range(100) {
    if i == 10 { break }
    if i % 2 == 1 { continue }
    total += i
}
print(total)      # 0 + 2 + 4 + 6 + 8 = 20
```

Everything is a statement

Control-flow constructs are statements, not expressions — they do not produce a value. To choose a value, assign inside the branches, or use `or` for a default (Chapter 6). This keeps control flow explicit and readable.

The next chapter covers the unit that organizes all this logic: the function.

Chapter 9 — Functions

Functions are the primary unit of organization in Sandy. They are also first-class values: you can store them, pass them, and return them.

Defining and calling

A function is introduced with `fn`, a name, a parenthesized parameter list, and a brace-delimited body:

```
fn add(a, b) {  
    return a + b  
}  
  
print(add(2, 3))    # 5
```

`return` yields a value and ends the function. A function that falls off the end, or uses a bare `return`, yields `nil`.

Parameters and return types

Parameters and the return may be annotated (Chapter 14). The annotations are optional and are checked before the program runs:

```
fn area(w: int, h: int) -> int {  
    return w * h  
}
```

Arguments are passed by value for scalars and strings, and by reference for lists, maps, and structs — that is, the callee receives the same container the caller has, so mutations are visible to both (see Chapter 11 for the details).

Recursion

A function may call itself. Names are resolved so that a function can refer to itself and to functions defined later in the same file:

```
fn fib(n) {  
    if n < 2 { return n }  
    return fib(n - 1) + fib(n - 2)  
}  
print(fib(10))    # 55
```

First-class functions

A function is a value. Assign it, pass it, return it:

```
fn square(n) { return n * n }

apply = square
print(apply(6))           # 36

fn twice(f, x) { return f(f(x)) }
fn inc(n) { return n + 1 }
print(twice(inc, 10))     # 12
```

This is what makes higher-order library functions like `lists.map`, `lists.filter`, and `lists.reduce` possible (Chapter 17).

Function types

When you want to *annotate* a function-valued parameter, the type is written `fn(ParamTypes) -> ReturnType`:

```
fn apply(f: fn(int) -> int, x: int) -> int {
    return f(x)
}
fn double(n: int) -> int { return n * 2 }

print(apply(double, 21))    # 42
```

A bare `fn` means "any function" and stays fully dynamic. With a precise signature, the checker verifies calls made through the value (arity and argument types). Typed top-level functions used as values also compile natively (Chapter 16).

Closures

A function defined inside another function *closes over* the variables in scope where it was defined — it can read and update them even after the outer function has returned. This makes stateful helpers and factories easy:

```
fn make_counter() {
    count = 0
    fn tick() {
        count += 1
        return count
    }
}
```

```
    return tick
}

next = make_counter()
print(next())    # 1
print(next())    # 2
print(next())    # 3
```

Each call to `make_counter` creates a fresh `count`, so two counters are independent.

Closures run on the interpreter and VM. The native compiler supports first-class *top-level* functions but not capturing closures (Chapter 16); the checker and interpreter support them everywhere.

Order of definitions

Top-level functions may be *called* before their definition appears in the file (calls are resolved after the whole file is read). But using a function as a *value* — assigning it, passing it — requires its definition to have run first, because that is when the function value comes into existence. In practice, define before you pass.

The next chapter covers the two workhorse container types: lists and maps.

Chapter 10 — Collections

Sandy has two built-in containers: the **list**, an ordered sequence, and the **map**, a key-value table. Both are mutable and grow on demand.

Lists

Creating and reading

```
xs = [10, 20, 30]
print(xs[0])      # 10
print(xs[-1])     # 30    (negative indexes count from the end)
print(len(xs))    # 3
```

Indexing out of range is a runtime error (catchable with `try`, Chapter 12).

Growing and mutating

```
xs = [1, 2, 3]
push(xs, 4)        # append; xs is now [1, 2, 3, 4]
xs[0] = 99         # replace an element
last = pop(xs)     # remove and return the last element
print(last)        # 4
```

Iterating

```
for x in [1, 2, 3] {
    print(x * x)    # 1, 4, 9
}
```

Building a list

A common pattern is to start empty and `push`:

```
squares = []
for i in range(1, 6) {
    push(squares, i * i)
}
print(squares)     # [1, 4, 9, 16, 25]
```

Concatenation and repetition

```
print([1, 2] + [3, 4])    # [1, 2, 3, 4]
print([0] * 3)            # [0, 0, 0]
```

Equality

Lists compare element-by-element:

```
print([1, 2, 3] == [1, 2, 3])    # true
```

Typed lists

Annotate a list with an element type to have it checked, and to make it eligible for native compilation:

```
nums: list<int> = [1, 2, 3]
```

The `lists` standard-library module adds higher-order helpers — `map`, `filter`, `reduce`, `reverse`, `unique`, and more (Chapter 17). The `sort` module sorts.

Maps

Creating and reading

```
ages = {"ada": 36, "bob": 41}
print(ages["ada"])    # 36
print(has(ages, "cy")) # false
```

Looking up a missing key is a runtime error; guard with `has`, or use `maps.get(m, key, default)` from the standard library.

Adding and updating

```
ages = {"ada": 36}
ages["bob"] = 41    # add a new entry
ages["ada"] = 37    # update an existing one
print(len(ages))    # 2
```

Keys, values, and iteration

Maps preserve **insertion order**. Iterating a map yields its keys:

```
m = {"a": 1, "b": 2, "c": 3}
for k in m {
    print(k + " = " + str(m[k]))
}
print(keys(m))      # ["a", "b", "c"]
print(values(m))    # [1, 2, 3]
```

Equality

Maps compare by their key-value pairs, regardless of order:

```
print({"a": 1, "b": 2} == {"b": 2, "a": 1})  # true
```

Typed maps

Annotate with key and value types:

```
scores: map<string, int> = {}
```

For native compilation, keys are `int` or `string` and values are scalars or strings.

Map helpers

The `maps` module (Chapter 17) adds `get`, `items`, `from_pairs`, `merge`, `invert`, `map_values`, and `pick`.

Choosing between them

Use a **list** for an ordered collection you iterate or index by position; use a **map** when you look values up by a key. Both are covered further — with worked examples — in the standard-library chapter.

The next chapter introduces the way to define your own structured types.

Chapter 11 — Structs

A struct is a user-defined type: a named group of fields. Structs give a program its vocabulary — `Point`, `Account`, `Request` — instead of loose maps and tuples.

Defining a struct

```
struct Point { x: int, y: int }
```

Fields may be typed (as above) or untyped:

```
struct Tag { name, weight }      # untyped fields – dynamic
```

Typed fields are checked and are required for native compilation. Field types may be scalars, strings, other structs, or (natively) typed lists and maps.

Creating an instance

Construct with the struct's name, passing one argument per field in order:

```
p = Point(3, 4)
print(p.x)      # 3
print(p.y)      # 4
```

Printing a struct shows its name and fields:

```
print(p)        # Point(x=3, y=4)
```

Reading and mutating fields

Fields are read and assigned with `.`:

```
p.x = 10
p.y += 5
print(p)        # Point(x=10, y=9)
```

Value equality

Two struct instances are equal when they have the same type and equal fields, compared deeply:

```
print(Point(1, 2) == Point(1, 2))    # true
print(Point(1, 2) == Point(1, 9))    # false
```

Reference semantics

Struct instances are **reference values**. Assigning one to another name, or passing one to a function, shares the *same* instance — mutations are visible everywhere it is referenced:

```
struct Box { n: int }

a = Box(1)
b = a          # b refers to the same Box as a
b.n = 42
print(a.n)     # 42 - a and b are the same instance

fn bump(box) { box.n += 100 }
bump(a)
print(a.n)     # 142 - the function mutated the shared instance
```

This is the same model Python uses for objects: names are references, and mutation is shared. If you need an independent copy, construct a new instance.

By contrast, scalars (`int` , `float` , `bool`) and strings are passed by value — a function cannot change a caller's number.

Nesting

A struct field may be another struct, and structs nest to any depth:

```
struct Point { x: int, y: int }
struct Line { a: Point, b: Point }

seg = Line(Point(0, 0), Point(3, 4))
print(seg.b.x)    # 3
print(seg)        # Line(a=Point(x=0, y=0), b=Point(x=3, y=4))
```

Structs as types

A struct name is a valid type annotation. The checker then verifies construction (arity and field types), field access (an unknown field is an error), and assignment — all before the program runs:

```
fn distance_sq(p: Point) -> int {
    return p.x * p.x + p.y * p.y
}
```

```
}
```

Structs — including those with list and map fields — compile natively (Chapter 16), where they become heap-allocated C structs that preserve exactly this reference semantics.

The next chapter covers what happens when things go wrong: error handling.

Chapter 12 — Error Handling

When something goes wrong at runtime — a division by zero, a missing map key, or a condition your own code decides is invalid — Sandy raises an *error*. An error unwinds the call stack until a `try` block catches it, or, if none does, stops the program with a message.

try / catch

Wrap code that might fail in `try`, and handle failure in `catch`:

```
try {
  x = int("not a number")
  print(x)
} catch e {
  print("failed: " + e)
}
```

```
failed: cannot convert 'not a number' to int
```

The name after `catch` is bound to the error **message**, which is always a string. That is the whole value of an error in Sandy — a human-readable message — which keeps error handling simple.

throw

Raise your own error with `throw`. The thrown value is converted to a string and becomes the caught message:

```
fn withdraw(balance, amount) {
  if amount > balance {
    throw "insufficient funds"
  }
  return balance - amount
}

try {
  print(withdraw(100, 250))
} catch e {
  print("declined: " + e)      # declined: insufficient funds
}
```

You can throw any value; it is stringified. In practice, throw a descriptive string.

Built-in errors are catchable

Errors raised by the language and its builtins are ordinary catchable errors — there is no separate category. Division by zero, an out-of-range index, a missing map key, and a failed conversion can all be caught:

```
fn safe_get(xs, i) {
  try {
    return xs[i]
  } catch e {
    return nil
  }
}
print(safe_get([1, 2, 3], 10))  # nil
```

The messages match what the language reports — for example "index 10 out of range (length 3)" and "division by zero".

Propagation

An error propagates outward through function calls until it meets a `try`. This lets you handle failures far from where they occur:

```
fn parse_row(line) {
  return int(line)      # may throw
}

fn total(lines) {
  sum = 0
  for line in lines {
    sum += parse_row(line)
  }
  return sum
}

try {
  print(total(["1", "2", "oops", "4"]))
} catch e {
  print("bad data: " + e)      # bad data: cannot convert 'oops' to int
}
```

`parse_row` doesn't handle the error; `total` doesn't either; the top-level `try` does.

What is *not* caught

`try / catch` catches runtime errors. It does not catch control flow — a `return`, `break`, or `continue` inside a `try` body does the ordinary thing (returns, breaks, continues); it is not intercepted by the `catch`.

In compiled programs

`try / catch / throw` compiles natively (Chapter 16). The compiled program uses the same semantics — the caught value is the message string, and built-in errors are catchable — so error-handling code behaves identically whether interpreted or compiled.

The next chapter covers organizing code across files and sharing it as packages.

Chapter 13 — Modules and Packages

A program larger than one file is organized into **modules**, and code shared between projects is distributed as **packages**. Both use the same `import`.

Importing a file

`import` runs another `.sy` file once and binds its top-level definitions — functions, structs, and variables — as a namespaced module value:

```
# geometry.sy
fn area(w, h) { return w * h }
fn perimeter(w, h) { return 2 * (w + h) }
```

```
# main.sy
import "geometry" as geo

print(geo.area(3, 4))      # 12
print(geo.perimeter(3, 4)) # 14
```

Access a module's members with `module.member`. The `.sy` extension is optional in the path (`"geometry"` finds `geometry.sy`).

Resolution and semantics

- Paths resolve **relative to the importing file**, then against installed package dependencies, then against the bundled standard library.
- A module is **cached by absolute path** — imported once, even through diamond import graphs where several files import the same module.
- **Circular imports** are detected and reported rather than looping.
- A module runs on the **same engine** as its importer, so a callback you pass into a module function behaves identically on the interpreter and the VM.

A local file always shadows a package, which shadows the standard library. This lets you override any module by dropping a file of the same name next to your program.

The standard library

The standard library is a set of modules, written in Sandy itself, imported by bare name from anywhere:

```
import "math" as math
import "json" as json

print(math.gcd(48, 36))          # 12
print(json.to_json({"ok": true})) # {"ok":true}
```

Chapter 17 documents every module.

Packages

A **project** is a directory with a manifest, `sandy.toml` :

```
[package]
name = "myapp"
version = "0.1.0"

[dependencies]
geometry = "^1.2.0"          # a registry version constraint
utils    = { path = "../utils" }    # a local path dependency
webby    = { git = "https://example.com/webby.git" } # a git dependency
```

The commands

```
sandy add geometry ^1.2.0    # add a dependency (version, path, or git URL)
sandy install                # resolve dependencies, vendor them, write sandy.lock
sandy publish                # publish this project to the registry
```

`sandy install` resolves every dependency, copies it into `sandy_modules/`, and records exactly what was chosen in `sandy.lock` so builds are reproducible. Dependencies are then imported by bare name, just like the standard library.

Version constraints

Registry dependencies use semantic-version constraints:

Constraint	Matches
<code>1.2.3</code>	exactly that version
<code>^1.2.0</code>	compatible: <code>>= 1.2.0</code> and <code>< 2.0.0</code>
<code>~1.2.0</code>	<code>>= 1.2.0</code> and <code>< 1.3.0</code>
<code>>=1.0.0,<2.0.0</code>	a range (comma-separated and)

Constraint	Matches
*	any version

`sandy install` picks the highest published version that satisfies the constraint.

The registry

The registry can be a local directory or an HTTP server, selected with the `SANDY_REGISTRY` environment variable (default `~/.sandy/registry`). Sandy includes a reference registry server:

```
sandy registry serve --port 8377 --dir ./registry
export SANDY_REGISTRY=http://localhost:8377
```

Published versions are immutable. Because dependencies are ordinary Sandy code, **typed dependencies are type-checked across the package boundary** — calling a dependency's function with the wrong argument type is caught before your program runs, even though it lives in another package.

The next chapter goes deep on the type system that makes that possible.

Chapter 14 — The Type System

Sandy's type system is **gradual**: annotations are optional, and code without them is fully dynamic and runs unchanged. Where you add annotations, a checker verifies them *before* the program runs. This chapter explains what the checker does, what it deliberately does not do, and how to run it.

The core promise

Unannotated code produces **zero** type errors and behaves exactly as if the checker did not exist. Annotations only ever *add* checks; they never change runtime behavior.

This is what "gradual" means. You can write an entire program with no types, add a few where a bug would be costly, and never be forced to annotate the rest.

any

Every unannotated value has the type `any`. `any` is compatible with every type, in both directions — you can pass an `any` where an `int` is expected, and an `int` where `any` is expected. This is the mechanism that keeps dynamic code error-free:

```
fn double(x: int) -> int { return x * 2 }
fn get(m, k) { return m[k] }           # untyped -> returns any

print(double(get(m, "n")))             # any flows into int: no error
```

Where annotations go

```
fn add(a: int, b: int) -> int { return a + b } # parameters and return
score: int = 0                                # variables
struct Point { x: int, y: int }                # struct fields
apply: fn(int) -> int = double                 # function-typed variables
```

What is checked

The checker reports an error only when two **known, non-`any`** types definitely conflict. Specifically it verifies:

- **Assignment** to an annotated variable, and later reassignment, match the declared type.
- **Return** values match a function's declared return type.

- **Call arity and argument types** against annotated parameters — including calls made through a value of function type `fn(...) -> R`.
- **Operators** — e.g. `int + string` is rejected when both sides are known.
- **List and map element types** via `list<T>` and `map<K, V>`; indexing a `list<string>` yields a `string`, so `xs[0] + 1` is caught.
- **Structs** — construction (arity and field types), field access (an unknown field is an error), and field assignment.
- **Imported modules** — calls to a module's functions and its struct construction are checked across the file (and package) boundary.
- **Unknown type names** in annotations are reported.

```
fn area(w: int, h: int) -> int { return w * h }
print(area("wide", 3))
```

```
TypeError (line 2): argument 1 of area() expects int, got string
```

Parameterized types

Lists and maps take type parameters:

```
nums: list<int> = [1, 2, 3]
table: map<string, int> = {"a": 1}
```

An unparameterized `list` or `map` is treated as fully gradual (element type `any`). Element and value types are checked when known, and `any` on either side always passes.

Function types

`fn(T1, T2) -> R` is the type of a function taking `T1, T2` and returning `R`; omit `-> R` for a function that returns nothing. A bare `fn` means "any function" and stays gradual. Calls made through a typed function value are checked for arity and argument types.

Int widens to float

An `int` is accepted where a `float` is expected (it widens automatically); the reverse is not allowed without an explicit `int(x)`:

```
x: float = 3          # fine – int widens to float
```

Running the checker

The checker runs automatically before `sandy run` and `sandy build`. To check without running:

```
sandy check program.sy
```

```
program.sy: no type errors ✓
```

Use `--no-check` to skip the pre-run check (for quick throwaway runs). The compiler always type-checks, because it needs the types to generate code.

Types and speed

The annotations are not only for safety — they drive the native compiler. When the typed subset of a program is compiled (Chapter 16), the known types let the compiler emit unboxed C: an `int` becomes a machine integer, a `list<int>` becomes a contiguous array, with no runtime type dispatch. The same annotations that catch your bugs make your code fast.

The next chapter covers doing several things at once.

Chapter 15 — Concurrency

Sandy's concurrency model is small and Go-flavored: run work as **tasks**, and have tasks communicate over **channels**. The guiding principle is *share memory by communicating* — pass data through channels rather than reaching into shared variables.

Spawning a task

`spawn(fn, args...)` runs a function as a background task and returns a handle. `wait(task)` blocks until the task finishes and returns its result:

```
fn work(n) {  
    return n * n  
}  
  
t = spawn(work, 12)  
print(wait(t))      # 144
```

If the task raises an error, `wait` re-raises it, so you can wrap `wait` in a `try`:

```
fn boom() { throw "kaboom" }  
b = spawn(boom)  
try { wait(b) } catch e { print("task failed: " + e) } # task failed: kaboom
```

Channels

A channel carries values between tasks. Create one with `channel()`, then `send` and `recv`:

```
ch = channel()  
  
fn producer() {  
    send(ch, 42)  
}  
spawn(producer)  
  
print(recv(ch))      # 42
```

`send` and `recv` **block to synchronize**:

- `channel()` — an **unbuffered** channel: a `send` waits until a `recv` takes the value (a rendezvous), so the two tasks hand off directly.
- `channel(n)` — a **buffered** channel holding up to `n` values before a sender blocks.

Because the operations block, values come out in a well-defined order even though the work runs concurrently — which is what makes concurrent programs testable and predictable.

Closing a channel

`close(ch)` marks a channel finished. A `recv` on a drained, closed channel returns `nil`, which gives a clean way to stream results and stop:

```
stream = channel()

fn emit() {
    for k in range(1, 4) {
        send(stream, k)
    }
    close(stream)
}
spawn(emit)

v = recv(stream)
while v != nil {
    print(v)          # 1, 2, 3
    v = recv(stream)
}
```

A worker pool

Putting it together — fan several tasks out and collect their results through one channel:

```
jobs = [1, 2, 3, 4, 5]
results = channel(len(jobs))

fn work(n) {
    send(results, n * n)
}
for n in jobs {
    spawn(work, n)
}

total = 0
i = 0
while i < len(jobs) {
    total += recv(results)
    i += 1
}
print(total)          # 55
```

Safety

Channels are internally synchronized, so passing values through them is always safe. Sharing a plain variable between tasks *without* a channel is a data race — exactly as in Go. The rule of thumb: **communicate through channels**, and reach for shared state only when you know what you're doing.

Where it runs

Tasks are real threads. `spawn` runs on the default engine (the interpreter); `channel`, `send`, `recv`, and `close` work everywhere. Because concurrency is inherently dynamic, it is not part of the native-compilation subset — concurrent programs run on the interpreter.

The next chapter turns to the other end of the spectrum: compiling for speed.

Chapter 16 — Compilation and Performance

Sandy runs the same source three ways. This chapter explains each, what compiles natively, and the performance you can expect.

Three engines, one language

Engine	Command	What it's for
Tree-walking interpreter	<code>sandy run</code>	development — instant, no build
Bytecode VM	<code>sandy --vm run</code>	a faster interpreter; behavior-identical
Native compiler	<code>sandy build</code>	production speed for the typed subset

The interpreter and VM run **all** of Sandy, including closures, dynamic values, and concurrency. The VM compiles to bytecode and runs a stack machine; it is kept output-identical to the interpreter by an equivalence test suite.

An honest baseline

The interpreter and VM are themselves written in Python. A Python program interpreting Sandy is necessarily *slower* than Python running Python — so on raw throughput the interpreter and VM lose to CPython. They exist for the edit-run loop, not for speed. All of Sandy's performance advantage comes from the native compiler.

Native compilation

`sandy build program.sy` translates the **statically typed subset** of Sandy to C and compiles it with the system C compiler (`cc` / `gcc` / `clang`) at `-O2` , producing a standalone executable:

```
sandy build fib.sy          # produces ./fib
sandy build fib.sy --run    # build and run
sandy build fib.sy -o app   # choose the output name
sandy build fib.sy --emit-c # also keep the generated C
sandy build fib.sy --gc     # link a garbage collector (see below)
```

What compiles

The native backend handles the typed core, which is most of the language:

- scalars (`int` , `float` , `bool` , `string`) and all arithmetic, comparison, and logic;

- typed **lists** `list<T>` and **maps** `map<K, V>` as unboxed C arrays and hash tables;
- **structs** with typed fields (including list and map fields), with value equality and reference semantics preserved;
- **functions** with typed parameters and a return type, including recursion and first-class *top-level* functions;
- all **control flow**, and **try / catch / throw**;
- `print`, string operations, and the numeric builtins.

What does not

Features that are inherently dynamic run on the interpreter/VM instead, and the compiler rejects them with a clear message: dynamic `any` values, capturing closures (top-level functions are fine), the OS/network builtins, and concurrency. When `sandy build` refuses, it tells you exactly why and points you to `sandy run`.

Types are the enabler

The compiler needs to know a value's type to emit unboxed code — an `int` as a machine integer, a `list<int>` as a contiguous array — with no runtime type tags. That is why the same annotations that make your code *safe* (Chapter 14) also make it *fast*. Untyped code cannot be compiled natively, because there is nothing for the compiler to specialize.

Memory

By default, native programs never free heap memory — which is fine for the short-lived tools the backend targets, and is the fastest option. For a long-running program, `sandy build --gc` links a conservative mark-sweep garbage collector, so memory stays bounded. Output and behavior are identical either way.

The numbers

On compute-bound benchmarks (CPython 3.11, `cc -O2`), native Sandy runs **15–180× faster than CPython**:

Benchmark	Python	Sandy native	Speedup
<code>fib(30)</code> recursive	138 ms	3.2 ms	43×
count primes <code>< 40k</code>	61 ms	3.9 ms	16×
<code>sum i*i</code> to 2,000,000	293 ms	1.6 ms	183×

The exact numbers vary by machine; the ratios are the point. The tight numeric loop shows the largest gap, where a compiler pulls furthest ahead of an interpreter. Reproduce it with `python bench/compare.py`,

and see `BENCHMARKS.md` for the methodology (the Sandy and Python versions are checked to compute the same result, so the comparison is fair).

The recommended workflow

1. Write and iterate with `sandy run` — instant feedback, full language.
2. Add types where they matter, and let `sandy check` guard them.
3. When you need speed, `sandy build` the typed program into a binary that runs at C speed.

You never leave the language or rewrite in C or Rust to get there. That is the lane Sandy is built for.

The remaining chapters are references: the standard library, the builtins, the tooling, and the grammar.

Chapter 17 — The Standard Library

Sandy ships with a standard library of modules, **written in Sandy itself** and imported by bare name. A local file of the same name shadows a standard module, so you can override any of them. This chapter documents each module.

```
import "math" as math
import "json" as json
# ... etc.
```

math — numbers and statistics

Constants `pi`, `e`, `tau`, and:

Function	Result
<code>sign(x)</code>	<code>-1</code> , <code>0</code> , or <code>1</code>
<code>clamp(x, lo, hi)</code>	<code>x</code> limited to the range
<code>gcd(a, b)</code> / <code>lcm(a, b)</code>	greatest common divisor / least common multiple
<code>factorial(n)</code>	<code>n!</code>
<code>is_prime(n)</code>	primality test
<code>mean(xs)</code> / <code>median(xs)</code>	average / middle value
<code>variance(xs)</code> / <code>stddev(xs)</code>	spread
<code>hypot(a, b)</code>	<code>sqrt(a*a + b*b)</code>
<code>deg2rad(d)</code> / <code>rad2deg(r)</code>	angle conversions
<code>log_base(x, base)</code>	logarithm in an arbitrary base

```
print(math.gcd(48, 36))      # 12
print(math.median([3, 1, 2])) # 2
```

(Trigonometric and exponential functions — `sin`, `cos`, `tan`, `exp`, `log`, `log10`, `sqrt`, `pow` — are global builtins; see Chapter 18.)

strings — string helpers

`reverse`, `capitalize`, `repeat(s, n)`, `pad_left(s, width, ch)`, `pad_right(s, width, ch)`, `count(s, sub)`, `contains(s, sub)`.

```
import "strings" as strings
print(strings.reverse("abc"))    # cba
print(strings.pad_left("7", 3, "0")) # 007
```

text — characters and tokenizing

Character predicates take a one-character string: `is_digit`, `is_alpha`, `is_alnum`, `is_space`, `is_upper`, `is_lower`. Plus `chars(s)` (list of characters), `lines(s)`, `words(s)` (split on whitespace runs), `title(s)` (capitalize each word), and `count(s, sub)`.

```
import "text" as text
print(text.words(" a b c ")) # ["a", "b", "c"]
print(text.title("hello there")) # Hello There
```

lists — higher-order list operations

`map(f, xs)`, `filter(pred, xs)`, `reduce(f, acc, xs)`, `reverse(xs)`, `unique(xs)`, `contains(xs, x)`, `index_of(xs, x)`, `take(xs, n)`, `drop(xs, n)`, `first(xs)`, `last(xs)`.

```
import "lists" as lists
fn even(n) { return n % 2 == 0 }
print(lists.filter(even, [1, 2, 3, 4])) # [2, 4]
```

sort — sorting

`sort(xs)`, `sort_by(xs, key)`, `sort_desc(xs)`, `is_sorted(xs)`. A stable, non-destructive insertion sort over numbers or strings.

```
import "sort" as sort
print(sort.sort([3, 1, 2])) # [1, 2, 3]
fn length(s) { return len(s) }
print(sort.sort_by(["ccc", "a", "bb"], length)) # ["a", "bb", "ccc"]
```

sets — set operations over lists

`union`, `intersection`, `difference`, `symmetric_difference`, `is_subset`, `is_disjoint`, `contains`, `unique`. Sets are lists of unique values.

```
import "sets" as sets
print(sets.union([1, 2], [2, 3]))      # [1, 2, 3]
print(sets.intersection([1, 2, 3], [2, 4])) # [2]
```

maps — map helpers

`get(m, k, default)`, `items(m)` (list of `[key, value]` pairs), `from_pairs`, `merge(a, b)`, `invert(m)`, `map_values(m, f)`, `pick(m, keys)`.

```
import "maps" as maps
print(maps.get({"a": 1}, "z", 0))      # 0
print(maps.merge({"a": 1}, {"b": 2}))  # {"a": 1, "b": 2}
```

json — JSON encoding and decoding

`to_json(value)` encodes a Sandy value to a JSON string; `parse(text)` decodes JSON back into Sandy values.

```
import "json" as json
text = json.to_json({"name": "Sandy", "xs": [1, 2, 3]})
print(text)                        # {"name": "Sandy", "xs": [1, 2, 3]}
data = json.parse(text)
print(data["name"])                # Sandy
```

csv — comma-separated values

`parse(text)` returns a list of rows (each a list of string fields); `format(rows)` produces CSV text. RFC 4180-style quoting handles commas, newlines, and quotes inside fields. Also `parse_line` and `format_row`.

```
import "csv" as csv
rows = csv.parse("name,age\nAda,36")
print(rows[1][0])                  # Ada
```

regex — regular expressions

`test(pattern, s)`, `find(pattern, s)`, `find_all(pattern, s)`, `groups(pattern, s)`, `replace(pattern, s, repl)`, `split(pattern, s)`. Uses the host regex syntax (`\d`, `\w`, anchors, groups, quantifiers). `find` / `groups` return `nil` when there is no match; `replace` supports `\1` backreferences.

```
import "regex" as regex
print(regex.find_all("\\w+", "a, bb, ccc")) # ["a", "bb", "ccc"]
print(regex.replace("\\s+", "a  b", "_"))   # a_b
```

random — pseudo-random numbers

A seedable, deterministic generator: `seed(n)`, `next()` (float in `[0, 1)`), `randint(lo, hi)` (inclusive), `boolean()`, `choice(xs)`, `shuffle(xs)` (a new list), `sample(xs, k)` (k distinct elements).

```
import "random" as random
random.seed(42)
print(random.randint(1, 6))      # a repeatable "die roll" for that seed
```

time — clocks and durations

`epoch()` (wall-clock seconds), `monotonic()` (for measuring elapsed time), `since(start)`, `sleep_ms(ms)`, and `format(seconds)` for a human-readable duration.

```
import "time" as time
print(time.format(3723))          # 1h 02m 03s
```

os — paths (with filesystem builtins)

Path helpers `join(a, b)`, `basename(path)`, `dirname(path)`, `extension(path)`, `stem(path)`. These pair with the filesystem builtins `cwd`, `exists`, `is_file`, `is_dir`, `list_dir`, `make_dir`, `remove_file`, and `args` (Chapter 18).

```
import "os" as os
print(os.basename("a/b/c.sy"))    # c.sy
print(os.extension("photo.png")) # png
```

http — web requests

`get_json(url)` and `post_json(url, value)` fetch and decode JSON; `status(response)`, `body(response)`, `ok(response)` read a raw response from the `http_get` / `http_post` builtins. A non-2xx status or a transport failure throws.

```
import "http" as http
data = http.get_json("https://api.example.com/thing")
print(data["name"])
```

base64 — Base64 encoding

`encode(s)` and `decode(s)` over the UTF-8 bytes of a string.

```
import "base64" as base64
print(base64.encode("hello"))    # aGVsbG8=
```

hash — a non-cryptographic hash

`djb2(s)` — a fast, stable integer hash for buckets and checksums. For cryptographic digests use the global `sha256(s)` and `md5(s)` builtins.

assert — assertions for tests

`eq(actual, expected, msg)`, `neq`, `is_true(cond, msg)`, `is_false`. Each throws a descriptive message on failure, so you can write tests in Sandy itself.

```
import "assert" as assert
assert.eq(2 + 2, 4, "arithmetic")
```

The next chapter documents the global built-in functions these modules build on.

Chapter 18 — Built-in Functions

These functions are always available — no import needed. They are grouped here by purpose. Many string and collection builtins also work as methods (`s.upper()` , `xs.push(x)`); the reference notes where.

Core

Function	Description
<code>print(...)</code>	print the arguments separated by spaces, then a newline
<code>input(prompt?)</code>	read a line of text from standard input
<code>len(x)</code>	length of a string, list, or map
<code>type(x)</code>	the value's type name, as a string
<code>range(n)</code> / <code>range(a, b)</code> / <code>range(a, b, step)</code>	a sequence of integers to loop over

Conversions

Function	Description
<code>str(x)</code>	the display string of any value
<code>int(x)</code>	convert a number or numeric string to an integer (truncates floats)
<code>float(x)</code>	convert a number or numeric string to a float
<code>bool(x)</code>	the truthiness of a value
<code>ord(ch)</code>	the Unicode code point of a one-character string
<code>chr(n)</code>	the one-character string for a code point

Math

Function	Description
<code>abs(x)</code>	absolute value
<code>min(list)</code> / <code>max(list)</code>	smallest / largest element
<code>sum(list)</code>	sum of a list of numbers
<code>round(x, ndigits?)</code>	round to <code>ndigits</code> decimal places (default 0)

Function	Description
<code>sqrt(x)</code>	square root
<code>floor(x)</code> / <code>ceil(x)</code>	round down / up to an integer
<code>pow(a, b)</code>	<code>a</code> to the power <code>b</code>
<code>sin</code> , <code>cos</code> , <code>tan</code>	trigonometric functions (radians)
<code>exp(x)</code>	e to the power <code>x</code>
<code>log(x, base?)</code>	natural logarithm, or logarithm in <code>base</code>
<code>log10(x)</code>	base-10 logarithm

Lists and maps

Function	Description
<code>push(list, x)</code>	append <code>x</code> to a list (also <code>list.push(x)</code>)
<code>pop(list)</code>	remove and return the last element
<code>keys(map)</code>	a list of the map's keys, in insertion order
<code>values(map)</code>	a list of the map's values
<code>has(container, x)</code>	whether a map has a key (or a list/string contains <code>x</code>)

Strings

Function	Description
<code>upper(s)</code> / <code>lower(s)</code>	change case (also methods)
<code>trim(s)</code>	remove leading/trailing whitespace (also a method)
<code>split(s, sep?)</code>	split into a list (whitespace if <code>sep</code> omitted)
<code>join(list, sep)</code>	join a list of strings with a separator

Hashing and encoding

Function	Description
<code>sha256(s)</code> / <code>md5(s)</code>	hex cryptographic digest of the UTF-8 text
<code>base64_encode(s)</code> / <code>base64_decode(s)</code>	Base64 encode / decode

Files

Function	Description
<code>read_file(path)</code>	the whole file as a string
<code>read_lines(path)</code>	the file as a list of lines
<code>write_file(path, text)</code>	write text, replacing the file
<code>append_file(path, text)</code>	append text to the file

The operating system

Function	Description
<code>args()</code>	the program's command-line arguments (a list of strings)
<code>env(name, default?)</code>	an environment variable, or the default / <code>nil</code>
<code>cwd()</code>	the current working directory
<code>exists(p)</code> / <code>is_file(p)</code> / <code>is_dir(p)</code>	filesystem queries
<code>list_dir(p)</code>	a sorted list of a directory's entries
<code>make_dir(p)</code>	create a directory (and any parents)
<code>remove_file(p)</code>	delete a file
<code>now()</code>	wall-clock seconds since the epoch
<code>clock()</code>	monotonic seconds, for measuring elapsed time
<code>sleep(seconds)</code>	pause execution
<code>exit(code?)</code>	end the program with an exit code

Regular expressions

Function	Description
<code>re_test(pattern, s)</code>	whether the pattern matches anywhere
<code>re_find(pattern, s)</code>	the first match, or <code>nil</code>
<code>re_find_all(pattern, s)</code>	a list of all matches
<code>re_groups(pattern, s)</code>	capture groups of the first match, or <code>nil</code>
<code>re_replace(pattern, s, repl)</code>	replace matches (<code>\1</code> backreferences)

Function	Description
<code>re_split(pattern, s)</code>	split on the pattern

The `regex` module (Chapter 17) wraps these with friendlier names.

Concurrency

Function	Description
<code>spawn(fn, args...)</code>	run <code>fn</code> as a background task; returns a task
<code>wait(task)</code>	block for a task and return its result
<code>channel(capacity?)</code>	create a channel (0 / omitted = unbuffered)
<code>send(ch, value)</code>	send a value (may block)
<code>recv(ch)</code>	receive a value (blocks; <code>nil</code> when closed and drained)
<code>close(ch)</code>	close a channel

See Chapter 15 for the concurrency model.

Note for compiled programs

The native compiler supports the numeric, string, list/map, and core builtins. The OS-facing, networking, regex, hashing, and concurrency builtins are dynamic and run on the interpreter/VM; `sandy build` reports clearly if a program uses one.

The next chapter covers the command-line tools around the language.

Chapter 19 — Tooling and the CLI

Sandy ships with the tools a language needs to be pleasant to work in: a runner, a compiler, a type checker, a formatter, a language server, and a browser playground. All are invoked through `python -m sandy` (shown below as `sandy`).

The command line

<code>sandy</code>	start the interactive REPL
<code>sandy FILE.sy</code>	run a program (tree-walking interpreter)
<code>sandy run FILE.sy</code>	run a program (explicit)
<code>sandy --vm FILE.sy</code>	run on the bytecode VM
<code>sandy --no-check FILE.sy</code>	run without the pre-run type check
<code>sandy check FILE.sy</code>	type-check without running
<code>sandy build FILE.sy</code>	compile the typed subset to a native binary
<code>sandy fmt FILE.sy</code>	format a program in place
<code>sandy add NAME SPEC</code>	add a dependency (version, path, or git URL)
<code>sandy install</code>	resolve dependencies and write <code>sandy.lock</code>
<code>sandy publish</code>	publish the current project to the registry
<code>sandy registry serve</code>	run an HTTP registry server
<code>sandy lsp</code>	start the language server (stdio, for editors)
<code>sandy --version</code>	print the version
<code>sandy --help</code>	show help

Running

`sandy run` uses the tree-walking interpreter — instant, full language, no build step. `sandy --vm run` uses the bytecode VM, a faster interpreter that is kept behavior-identical. Both type-check first; pass `--no-check` to skip that for a quick throwaway run.

Arguments after the filename are passed to the program and read with `args()`:

```
sandy run tool.sy input.txt --verbose
```

Building

`sandy build` compiles the typed subset to a native executable (Chapter 16):

```
sandy build app.sy          # -> ./app
sandy build app.sy --run    # build and run
sandy build app.sy -o mytool # output name
```

```
sandy build app.sy --emit-c    # keep the generated C
sandy build app.sy --gc       # link the garbage collector
```

Checking

`sandy check` runs the type checker and reports, without executing:

```
sandy check app.sy
```

```
app.sy: no type errors ✓
```

The formatter

`sandy fmt` rewrites a file in Sandy's canonical style — consistent indentation, spacing, and precedence-correct parenthesization — while preserving your comments and blank lines. It is idempotent (formatting twice changes nothing) and semantics-preserving.

```
sandy fmt app.sy           # reformat in place
sandy fmt --check app.sy   # exit non-zero if not already formatted (for CI)
```

The language server

`sandy lsp` speaks the Language Server Protocol over stdio, so editors get live Sandy support: syntax and type-error diagnostics as you type, formatting, completion (keywords, builtins, and your file's own definitions), a symbol outline, hover signatures, and go-to-definition. It reuses the same lexer, parser, checker, and formatter as the CLI, so the editor and the command line always agree.

A ready-to-use VS Code extension, with a TextMate grammar for syntax highlighting, lives in `editors/vscode/`.

The REPL

Running `sandy` with no file starts a read-eval-print loop. Type an expression to see its value, or statements to run them, with errors reported inline — handy for exploring the language and the standard library.

The browser playground

`web/playground.html` runs the **real** Sandy implementation in the browser via WebAssembly (Pyodide) — no server, no install. Open it locally or host it on any static site to let people try Sandy from a link. It is generated from the package sources and kept in sync by a test, so it never drifts from the language.

Testing your own code

Because the `assert` module (Chapter 17) throws a descriptive message on failure, you can write tests as ordinary Sandy programs:

```
import "assert" as assert

fn add(a, b) { return a + b }

assert.eq(add(2, 3), 5, "adds")
assert.is_true(add(2, 3) > 0, "positive")
print("all tests passed")
```

Run them with `sandy run tests.sy` (or compile them). The language's own test suite additionally checks the interpreter and VM against each other and the native output against the interpreter.

The final chapter is the formal grammar.

Chapter 20 — Appendix: Grammar

This appendix gives the formal syntax of Sandy, the keyword and operator reference, and a compact summary of the language.

Grammar

Approximate EBNF (`*` = zero or more, `?` = optional, `|` = alternative). Newlines separate statements inside a block.

```
program      = statement* ;

statement    = funcdef | structdef | import | if | while | for
              | return | break | continue | try | throw | assign | exprstmt ;

funcdef      = "fn" IDENT "(" params? ")" ( "->" type )? block ;
params       = param ( "," param )* ;
param        = IDENT ( ":" type )? ;

structdef    = "struct" IDENT "{" ( field ","? )* "}" ;
field        = IDENT ( ":" type )? ;

import       = "import" STRING ( "as" IDENT )? ;

if           = "if" expr block ( "elif" expr block )* ( "else" block )? ;
while        = "while" expr block ;
for          = "for" IDENT "in" expr block ;
return       = "return" expr? ;
break        = "break" ;
continue     = "continue" ;
try          = "try" block "catch" IDENT block ;
throw        = "throw" expr ;

assign       = target ( "=" | "+=" | "-=" | "*=" | "/=" ) expr
              | IDENT ":" type "=" expr ;
target       = IDENT | postfix "[" expr "]" | postfix "." IDENT ;
exprstmt     = expr ;

block        = "{" statement* "}" ;

expr         = or ;
or           = and ( "or" and )* ;
and          = not ( "and" not )* ;
not          = "not" not | comparison ;
comparison   = additive ( ( "==" | "!=" | "<" | ">" | "<=" | ">=" ) additive )* ;
additive     = multiplicative ( ( "+" | "-" ) multiplicative )* ;
```

```

multiplicative = power ( ( "*" | "/" | "%" ) power ) * ;
power          = unary ( "***" power ) ? ;
unary          = "-" unary | postfix ;
postfix        = primary ( call | index | attr ) * ;
call           = "(" ( expr ( "," expr ) * ) ? ")" ;
index          = "[" expr "]" ;
attr           = "." IDENT ;
primary        = INT | FLOAT | STRING | "true" | "false" | "nil"
               | IDENT | "(" expr ")" | list | map ;
list           = "[" ( expr ( "," expr ) * "," ? ) ? "]" ;
map            = "{" ( entry ( "," entry ) * "," ? ) ? "}" ;
entry          = expr ":" expr ;

type           = "int" | "float" | "string" | "bool" | "nil" | "any"
               | "fn" ( "(" ( type ( "," type ) * ) ? ")" ( "->" type ) ? ) ?
               | "list" ( "<" type ">" ) ?
               | "map" ( "<" type "," type ">" ) ?
               | IDENT ;    (* a struct name *)

```

Operator precedence

From lowest (binds loosest) to highest (binds tightest):

Level	Operators	Associativity
1	<code>or</code>	left
2	<code>and</code>	left
3	<code>not</code> (unary)	right
4	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code>	left
5	<code>+</code> <code>-</code>	left
6	<code>*</code> <code>/</code> <code>%</code>	left
7	<code>**</code>	right
8	unary <code>-</code>	right
9	postfix <code>()</code> <code>[]</code> <code>.</code>	left

Keywords

Reserved words that cannot be used as identifiers:

```

fn  return  if  elif  else  while  for  in  break  continue
try  catch  throw  struct  import  true  false  nil  and  or  not

```


`as` is a soft keyword, meaningful only in `import "x" as name`.

Escape sequences in strings

```
\n    newline        \"    double quote
\t    tab           \\    backslash
\r    carriage return
{expr} interpolation  {{ literal {    }} literal }
```

Semantic quick reference

A few rules that are easy to forget:

- Statements are separated by **newlines**; there is no `;`.
- `elif` and `else` sit on the **same line** as the preceding `}`.
- `/` **always produces a float**; there is no integer-division operator (use `floor(a / b)`).
- `%` is **floor modulo** (the result has the sign of the divisor).
- `**` is exponentiation and is **right-associative**.
- Scope is **function-level**; loop and `if` blocks do not create a new scope.
- Structs, lists, and maps are **reference values**; scalars and strings are passed by value.
- A caught error is always a **string** (the message).
- `type()` returns the type name; struct instances report their struct's name.

The language in one screen

```
# variables, interpolation
name = "Sandy"
print("hello {name}")

# functions, optional types
fn add(a: int, b: int) -> int { return a + b }

# control flow
for i in range(3) {
    if i % 2 == 0 { print("{i} even") } else { print("{i} odd") }
}

# collections
xs = [1, 2, 3]
push(xs, 4)
m = {"a": 1}
m["b"] = 2

# structs
struct Point { x: int, y: int }
p = Point(1, 2)
```

```
p.y += 5

# errors
try { throw "nope" } catch e { print(e) }

# modules
import "math" as math
print(math.gcd(12, 18))

# concurrency
ch = channel()
fn worker() { send(ch, 42) }
spawn(worker)
print(recv(ch))
```

This is the end of the book. You now know all of Sandy: the syntax, the type system, the standard library, the tools, and how to compile it for speed. The [reference](#) is the terse companion, the [examples](#) are runnable, and the source is yours to read — Sandy is a small language, and reading its implementation is one of the best ways to understand it.